An Adaptive Framework for Concept Drift Detection in Intrusion Detection Systems

Supervisor:

Dr.A.John Prakash
Associate Professor

Presented By:

A.T.Pranav varshan
2025165034
M.E-CSE (DSCS)

Definition

Concept drift is the change in the underlying **data distribution** or the relationship between **input features** and **target labels** over time. It occurs in dynamic, **non-stationary** environments where data continuously evolves.

Mathematical Representation

$$P_t(Y \mid X) \neq P_{t+\Delta t}(Y \mid X)$$

where

- X : Input feature vector
- Y : Class label
- t : Current time
- Δt : Time interval

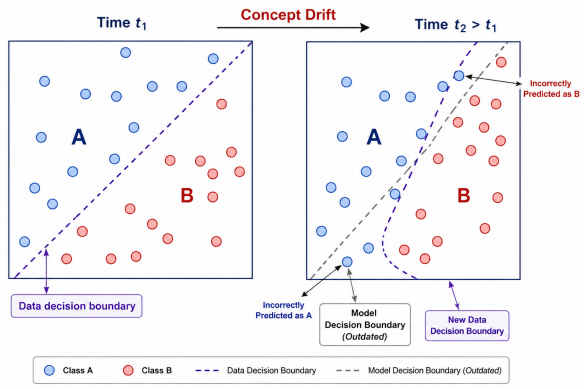


Fig. 1. Decision boundary shift due to concept drift.

Introduction: Significance of Concept Drift in Cybersecurity

- **Machine Learning (ML)** is widely adopted for **intrusion** detection, **malware** detection, and **threat analysis** in modern cybersecurity.
- Continuous evolution of cyber **threats**, **user behavior**, and **network** environments introduces **concept drift**, causing the deployment data distribution to differ from the training data.
- Consequently, **detection performance degrades**, making **concept drift detection** and **adaptive model updating** essential for maintaining reliable cybersecurity systems.

Overall Objectives

Phase 1

Main Objective

Develop a concept drift management framework for accurate detection, explanation, and adaptation to concept drift.

Specific Objectives

- Identify and prioritize significant drift samples for efficient model adaptation.
- Generate interpretable explanations for detected concept drift.
- Adapt the intrusion detection model through incremental learning using representative drift samples.

Phase 2

Main Objective

Develop a behavior-aware continual learning framework for continuous adaptation to evolving cyber threats without full retraining.

Specific Objectives

- Characterize behavioral patterns of drift samples to understand attack evolution.
- Leverage historical knowledge to distinguish recurring and emerging attack behaviors.
- Continuously update the intrusion detection model while mitigating catastrophic forgetting through continual learning.

Literature Survey (1/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	**Methodology / Technique / Algorithm Used**	**Issues / Gaps / Limitations**	**Ideas for Adoption**
Detecting and Rationalizing Concept Drift: A Feature-Level Approach **Authors:** Yang et al. **Year:** 2025 **Journal:** Expert Systems with Applications **Vol./Issue/PP:**260/ N/A /125365	Feature-level concept drift detection; feature importance analysis; cause-effect reasoning for identifying drift-causing features. The approach explains which features contribute to the detected drift and improves the interpretability of drift analysis.	No complete adaptive learning mechanism; limited real-time applicability; mainly focuses on explaining drift rather than adapting the detection model. The method identifies the cause of drift but does not automatically update the model.	Use feature-level drift explanation to identify which features cause drift and use the identified features to guide model adaptation. This can improve the interpretability of an adaptive cybersecurity system.

Literature Survey (2/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	**Methodology / Technique / Algorithm Used**	**Issues / Gaps / Limitations**	**Ideas for Adoption**
DriftTrace: Combating Concept Drift in Security Applications Through Detection and Explanation **Authors:** Authors as listed in reference **Year:** 2024 **Journal:** IEEE Transactions on Information Forensics and Security **Vol./Issue/PP:**21/ N/A /1957–1972	Contrastive feature alignment; greedy feature selection; sample interpolation; drift detection and explanation. The method aligns features between different distributions and identifies important drift-related features.	Complex architecture; higher training overhead; computationally expensive adaptation. The approach may require significant computational resources for large-scale security datasets.	Adopt contrastive learning and feature selection to detect, explain, and adapt to concept drift. The selected drift-related features can be used to trigger targeted model adaptation.

Literature Survey (3/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	Methodology / Technique / Algorithm Used	Issues / Gaps / Limitations	Ideas for Adoption
A Framework for Drift Detection and Adaptation in AI-driven Anomaly and Threat Detection Systems (HDDAF) **Authors:** Lara-Gutiérrez et al. **Year:** 2025 **Journal:** International Journal of Information Security **Vol./Issue/PP:** 24/ 5/ 199	Hoeffding-based drift detection; incremental learning; adversarial training; continuous model adaptation. The framework detects distribution changes and continuously updates the threat detection model.	Computationally expensive; complex implementation; continuous adaptation requires additional computational resources. The combination of multiple adaptation techniques increases system complexity.	Adopt online drift detection with incremental learning for continuous adaptation of cybersecurity models. This can reduce the need for complete model retraining after every drift event.

Literature Survey (4/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	**Methodology / Technique / Algorithm Used**	**Issues / Gaps / Limitations**	**Ideas for Adoption**
CADE: Detecting and Explaining Concept Drift Samples for Security Applications **Authors:** Yang et al. **Year:** 2021 **Conference:** USENIX Security Symposium **Vol./Issue/PP:** N/A/ N/A/ 2327–2344	Contrastive Autoencoder (CAE); distance-based drift detection; sample-level drift detection; feature-level explanation. The method detects individual drift samples and provides explanations for the observed feature changes.	No model adaptation mechanism; high computational cost; mainly focuses on detection and explanation. The detected drift samples are not automatically used to update the model.	Adopt Contrastive Autoencoder-based drift detection to identify and explain drift samples before model adaptation. The identified samples can be used as input for subsequent incremental retraining.

Literature Survey (5/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	**Methodology / Technique / Algorithm Used**	**Issues / Gaps / Limitations**	**Ideas for Adoption**
Robust Malicious Network Traffic Detection Framework With Automated Drift Detection, Identification, and Adaptation **Authors:** Authors as listed in reference **Year:** 2024/2025 **Journal:** IEEE Transactions on Information Forensics and Security **Vol./Issue/PP:** 21/ N/A/ 4985–5000	Contrastive Autoencoder; automated drift detection; clustering-based drift identification; model adaptation. The framework automatically detects drift, identifies the type or characteristics of the drift, and adapts the malicious traffic detection model.	High computational overhead; complex multi-stage framework; implementation complexity. Multiple processing stages can increase latency and resource requirements.	Adopt a **Detect** $\rightarrow$ **Identify** $\rightarrow$ **Adapt** pipeline for automatically responding to changing network traffic. This provides a structured approach for building an automated drift-aware IDS.

Literature Survey (6/6)

Paper Title, Authors, Year, Journal, Vol./Issue/PP	**Methodology / Technique / Algorithm Used**	**Issues / Gaps / Limitations**	**Ideas for Adoption**
Self-Supervised Adaptation Method to Concept Drift for Network Intrusion Detection **Authors:** Authors as listed in reference **Year:** 2024 **Journal:** IEEE Transactions on Dependable and Secure Computing **Vol./Issue/PP:** 22/ 6/ 7632–7646	Self-supervised representation learning; weakly supervised fine-tuning; concept-drift adaptation. The approach learns useful representations from unlabeled network data and adapts the detection model to changed data distributions.	High training complexity; focuses mainly on adaptation; adaptation strategy may require careful tuning. The approach can still require labelled information for effective fine-tuning.	Adopt self-supervised learning to reduce dependence on labelled data after concept drift. This can enable efficient adaptation when newly arrived network data has limited labels.

Summary of Issues

- **Limited Model Adaptation:** Many approaches detect and explain concept drift but do not provide **automatic model adaptation**.
- **High Computational Overhead:** Contrastive learning, autoencoders, incremental learning and adversarial training can be **computationally expensive**.
- **Complex Frameworks:** Multi-stage drift detection and adaptation pipelines increase **implementation and maintenance complexity**.
- **Limited Explainability:** Existing methods may identify the cause of drift but provide limited explanation of the **adaptation decision**.
- **Dependence on Labels and Retraining:** Several approaches require **labelled data or frequent retraining**, which is difficult in real-world cybersecurity environments.

Overall Research Gap:

A lightweight and unified framework is needed to combine **drift detection, explanation, and automatic adaptation** *with minimal labelled data and low computational overhead.*

Proposed System

- **Concept Drift Detection:** Learns robust feature representations to detect concept drift and distinguish normal traffic from evolving attack patterns.
- **Drift Explanation and Behaviour Analysis:** Explains detected drift and analyzes behavioural changes to derive meaningful attack context.
- **Knowledge Evolution:** Integrates behavioural context with historical knowledge to refine and continuously update the knowledge base.
- **Continual Model Adaptation:** Incrementally adapts the intrusion detection model using behaviour-aware learning while preserving previously acquired knowledge.

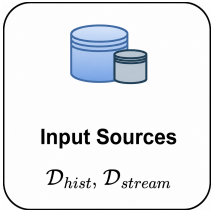
Input Sources

Objective

Collect benchmark cybersecurity datasets for feature learning and concept drift detection.

Datasets Used

- **CICIDS2018**
  - Network intrusion dataset
  - Benign and attack traffic
- **MalDroid2020**
  - Android malware dataset
  - Benign and malicious apps



Output: Combined dataset sent to the Data Preprocessing module.

Training Dataset

$$D_{train} = D_{CICIDS2018} \cup D_{MalDroid2020}$$

Data Preprocessing

Objective

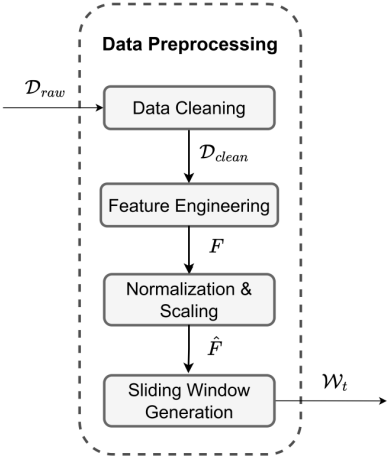
Prepare the input datasets by improving data quality, extracting meaningful features, standardizing feature values, and generating fixed-size data windows for contrastive feature learning.

Processing Steps

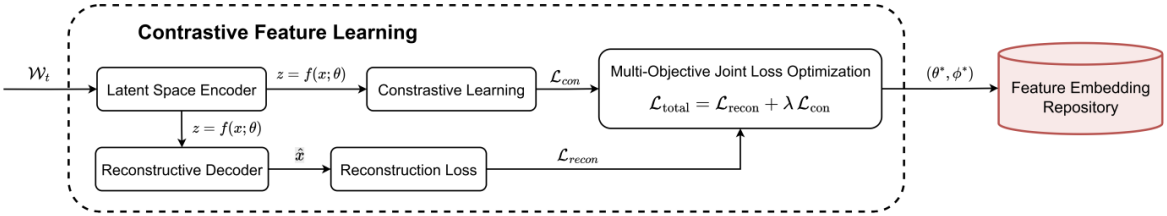
1. Data Cleaning
2. Feature Engineering
3. Normalization & Scaling
4. Sliding Window Generation

Output

- Clean and normalized feature vectors
- Fixed-size sliding windows (W_t)
- Ready for Contrastive Feature Learning



Contrastive Feature Learning



Objective

Learn compact and discriminative latent representations from scaled sliding-window batches by combining reconstruction and contrastive learning.

Processing Steps

1. Encoder–Decoder Construction
2. Latent Feature Extraction
3. Reconstruction Loss Calculation
4. Contrastive Loss Calculation

Output

- Optimized model parameters (θ^*, ϕ^*)
- Latent feature embeddings (Z)
- Feature Embedding Repository $(\mathcal{R}_{emb})$

Module 1: Data Preprocessing & Window Generation

Inputs: Raw historical dataset (D_{hist}) and streaming data (D_{stream}).

Outputs: Scaled sliding-window batches (W_t).

Algorithm / Pseudocode Steps:

1. **Initialize Datasets:** Load raw benchmark datasets (`IDS2018` and `MalDroid2020`).
2. **Data Cleaning:** Remove embedded headers and resolve `Infinity`/`NaN` values to prevent computation errors.
3. **Feature Engineering:** Remove non-behavioral attributes such as `Flow ID`, `Src IP`, `Dst IP`, `Src Port`, and `Timestamp`.
4. **Standardization:** Initialize `FeatureScaler`; fit only on the training split and transform the testing split to prevent **data leakage**.
5. **Window Generation:** Apply `SlidingWindowGenerator` with fixed window size (W) and step size to generate continuous streaming batches:

$$\mathcal{W}_t = (X_{t:t+W},\, y_{t:t+W})$$

Module 2: Contrastive Feature Learning

Inputs: Scaled sliding-window batches (W_t).

Outputs: Optimized parameters (θ^*, ϕ^*) and latent embeddings (Z) saved in the Feature Embedding Repository ($\mathcal{R}_{emb}$).

Algorithm / Pseudocode Steps:

1. **Initialize Model:** Build a Symmetric MLP Contrastive Autoencoder with a Latent Encoder $z = f(x; \theta)$ and Reconstructive Decoder $\hat{x} = h(z; \phi)$.
2. **Forward Pass:** Map high-dimensional feature vectors into a lower-dimensional latent space to obtain compact representations.
3. **Reconstruction Loss:** Compute Mean Squared Error (MSE) between the original input x and reconstructed output $\hat{x}$:

$$\mathcal{L}_{recon} = MSE(x, \hat{x})$$

4. **Contrastive Loss:** Compute pairwise contrastive loss $\mathcal{L}_{con}$ using margin $m = 10.0$ to bring similar classes closer and separate different classes in the latent space.
5. **Joint Loss Optimization:** Optimize the combined objective:

$$\mathcal{L}_{total} = \mathcal{L}_{recon} + \lambda \mathcal{L}_{con}, \qquad \lambda = 0.1$$

Implementation Details – Data Preprocessing

Data Preprocessing

- **Dataset Loading:** Benchmark dataset (`IDS2018`) loaded and `Infinity`/`NaN` values cleaned.
- **Feature Selection:** Non-behavioral features such as IP addresses, ports, and timestamps were removed.
- **Standardization:** Applied **zero-leakage standardization** using `FeatureScaler`.

Output

- Cleaned and standardized network features
- Scaled feature vectors ready for window generation

```
IDS2018 Non-Behavioral Columns Dropped : ['Flow ID', 'Src IP', 'Dst IP', 'Src Port', 'Timestamp']
IDS2018 Final Behavioral Feature Count : 78 features (83 raw -> 78 behavioral)
IDS2018 Label Encoding Map             : {'Benign': 0, 'DoS attacks-Hulk': 1, 'Bot': 2, 'Infilteration': 3}

MalDroid2020 Feature Count             : 470 dynamic syscall/binder frequencies
MalDroid2020 Label Encoding Map        : {5: 0, 1: 1, 2: 2, 4: 3, 3: 4}
```

Implementation Details – Contrastive Feature Learning

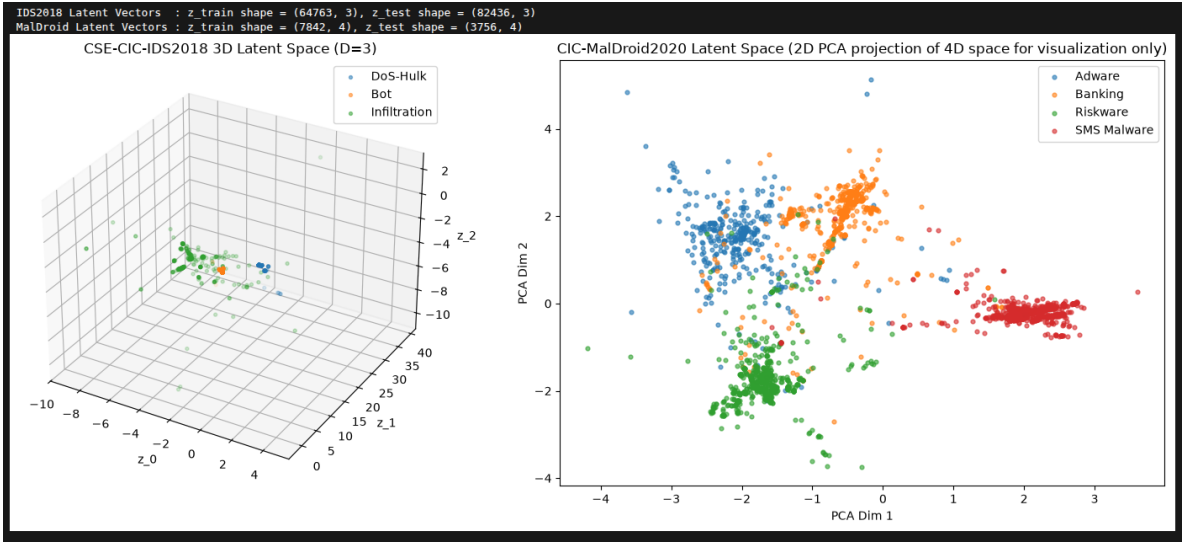
Contrastive Feature Learning

- **Symmetric MLP Autoencoder:** Reduced the input feature space from **78 features to 3 latent dimensions**.
- **Joint Loss Optimization:** Optimized the model using Reconstruction MSE (Eq. 1) and Contrastive Loss (Eq. 2).
- **Feature Embedding:** Generated discriminative latent representations for the input network traffic.

Output

- Optimized latent embeddings (Z)
- IDS2018: $Z_{\text{train}} \in \mathbb{R}^{64763\times 3}$
- MalDroid2020: $Z_{\text{train}} \in \mathbb{R}^{7842\times 4}$
- Stored in the Feature Embedding Repository ($\mathcal{R}_{emb}$)

Contrastive Feature Learning – Latent Space



Performance Evaluation Metrics (1/2)

Detection Performance

- **Accuracy (Acc):** Percentage of correctly classified network traffic over all samples.
- **Precision (Pre):** Measures how many detected attacks are truly malicious, reducing false alarms.
- **Recall (Rec):** Measures the ability to identify all actual attacks and drifted samples.
- **F1-Score (F1):** Harmonic mean of Precision and Recall; the primary metric for imbalanced intrusion datasets.

Concept Drift Evaluation

- **Drift Ratio (DR):** Percentage of incoming samples identified as concept drift, indicating drift severity.
- **Non-Drift Ratio (NDR):** Percentage of samples remaining within the learned distribution, reflecting model stability.

Performance Evaluation Metrics (2/2)

Explainability Evaluation

- **Latent Distance Reduction (LDR):** Measures how effectively the identified features explain drift by reducing latent-space distance to known classes.
- **Boundary Crossing Rate (BCR):** Percentage of drift samples that move back to a known class after modifying the explained features.

Deployment Performance

- **Inspection Effort (IE):** Number of samples requiring manual verification to identify true drift, reflecting analyst workload.
- **Execution Time & Memory Usage:** Measures computational latency and memory consumption to assess real-time deployment feasibility.

References I